Universidad Nacional Autónoma de México
Facultad de Ingeniería

Sistemas Operativos (sistop-2027-1)

Reporte Ploblema 5

Hernández Andrade, Miguel Ángel
Santos Cruz Lair Abraham

15 de octubre de 2026

1. El problema que decidimos resolver

Decidimos resolver el problema número 5 que es una simulación de un servidor web concurrente. El problema central consiste en manejar la llegada continua y asíncrona de conexiones de red y distribuirlas de manera eficiente entre un grupo de k hilos que se encargarán de "procesarlos". Elegimos este problema porque es un problema real y en la clase del 01/10/26 mencionaron un patrón de trabajo que podríamos utilizar.

2. Lenguaje y entorno de desarrollo

El programa fue desarrollado íntegramente en **Python** . Utilizamos Visual Studio Code como entorno de desarrollo y Git/GitHub para el control de versiones y la colaboración sincrónica entre ambos integrantes en la rama `Tarea1`.

2.1. Requisitos y ejecución en otra computadora

Para que nuestro programa funcione en su equipo, es necesario cumplir con lo siguiente:

- **Requisitos:** Tener instalado Python. No se requiere instalar librerías externas de terceros.
- **Ejecución:**
- **Nota de uso:** El sistema simula un servidor continuo que recibe tráfico permanentemente.

3. Estrategia de sincronización

Para resolver el problema, empleamos el patrón concurrente **Productor-Consumidor**, el cual modela perfectamente la interacción Jefe-Trabajador. Nuestra estrategia se apoya en tres elementos clave:

- **Cola compartida (FIFO):** Utilizamos una estructura `deque` de Python para almacenar las solicitudes entrantes, garantizando que se comiencen a procesar en estricto orden de llegada.
- **Mutex (`Semaphore(1)`):** Implementamos un semáforo binario para proteger la memoria compartida (la cinta de peticiones). Cada vez que el jefe añade una tarea o un trabajador la extrae, adquieren este mutex, evitando la corrupción de datos al operar sobre la cola simultáneamente.
- **Semáforo contador (`peticiones`):** Inicializado en 0. Este es nuestro mecanismo para la espera pasiva. Cuando la cola está vacía, los trabajadores se bloquean con un `acquire()` y se van a dormir. Cuando el jefe recibe una conexión, encola la tarea y ejecuta un `release()` para despertar inmediatamente a un trabajador disponible.

4. Refinamientos implementados

Atendiendo a las especificaciones del sistema, implementamos el requerimiento de **información de contabilidad**. En lugar de solo procesar la tarea en silencio, cada hilo trabajador calcula el tiempo exacto de procesamiento que le tomó resolver la petición (simulado mediante tiempos aleatorios). Al finalizar, cada trabajador notifica su rendimiento imprimiendo el nombre de la página despachada, el tiempo en segundos invertido en ella y su total acumulado de tareas completadas.

5. Dudas

Durante el desarrollo, nos encontramos con un dilema de diseño sobre el cual nos gustaría recibir retroalimentación:

- **Terminación limpia vs. Ejecución infinita:** Inicialmente construimos el código para procesar un número finito de solicitudes.Sin embargo, al observar el código base de clase, notamos que el servidor debía operar en un ciclo infinito continuo. Optamos por la ejecución infinita con reportes de rendimiento emitidos por tarea, pero tenemos la duda de si, para fines prácticos del curso, es preferible diseñar siempre estos servidores con un mecanismo de apagado limpio.